

University of Stuttgart
Germany

Client-server architecture model

With sockets and multiprocessing in Python

Ilche Georgievski
ilche.georgievski@iaas.uni-stuttgart.de

Descriptive models for distributed system design

Physical
model

Architectural model

Interaction
model

Architectural elements

Communicating
entities

Processes

Objects

Components

Web Services

Communication paradigm

Inter-process communication

UDP
sockets

TCP
sockets

Multi-
cast

Indirect
communication

Remote
invocation

Roles and res-
ponsibilities

Architectural
styles

Client-server

Peer-to-peer

Placement

Multiple server

Proxy/Cache

Mobile code

Interaction
model

Failure
model

Security
model

Architectural patterns

Vertical distribution

Multi-tier

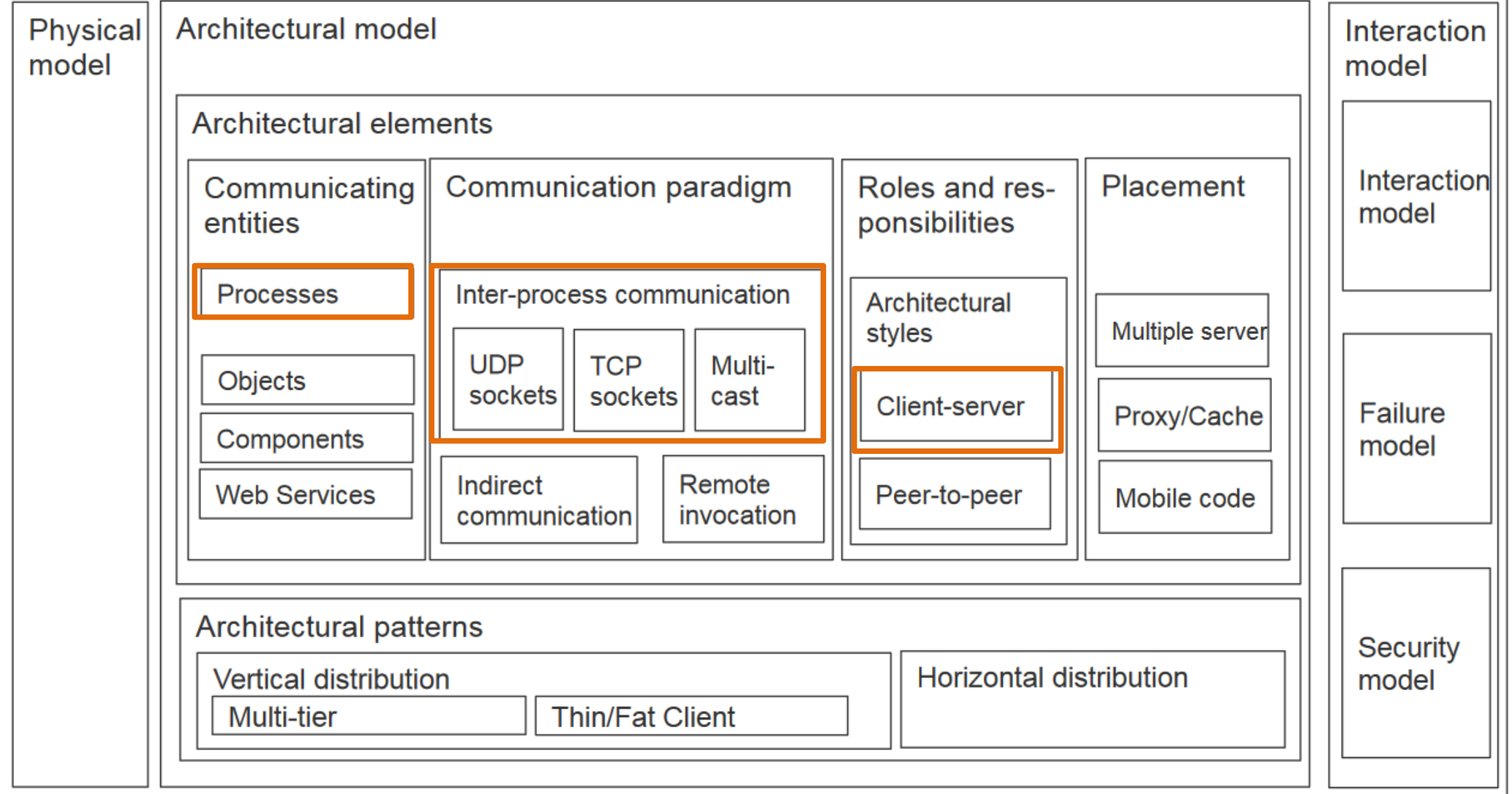
Thin/Fat Client

Horizontal distribution

Architecture models

- Client-server architecture model
 - Functionally distinctive components: *clients* and *servers*
 - Clients request services from servers
 - Centralised approach
- Peer-to-peer architecture model
 - Equipotent components called *peers*
 - Each peers can request services and respond to requests
 - Decentralised approach

Descriptive models for distributed system design



CLIENT-SERVER ARCHITECTURE MODEL

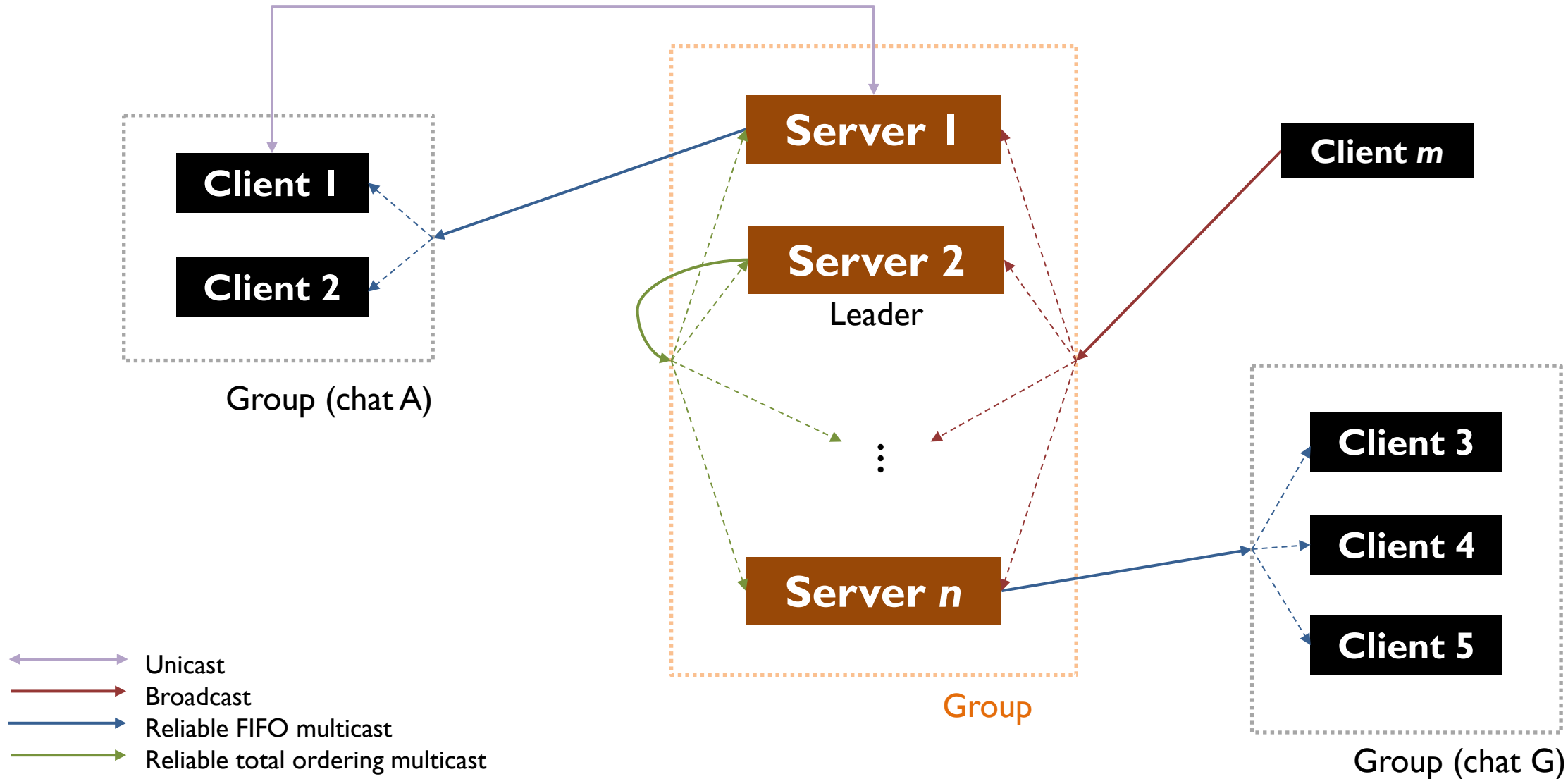
Client-server architecture model

- **Server**
 - Application that passively waits for contact
 - Runs on big machines
 - Serves many clients simultaneously
- **Client**
 - Application that actively initiates contact
 - Runs on end-users' devices

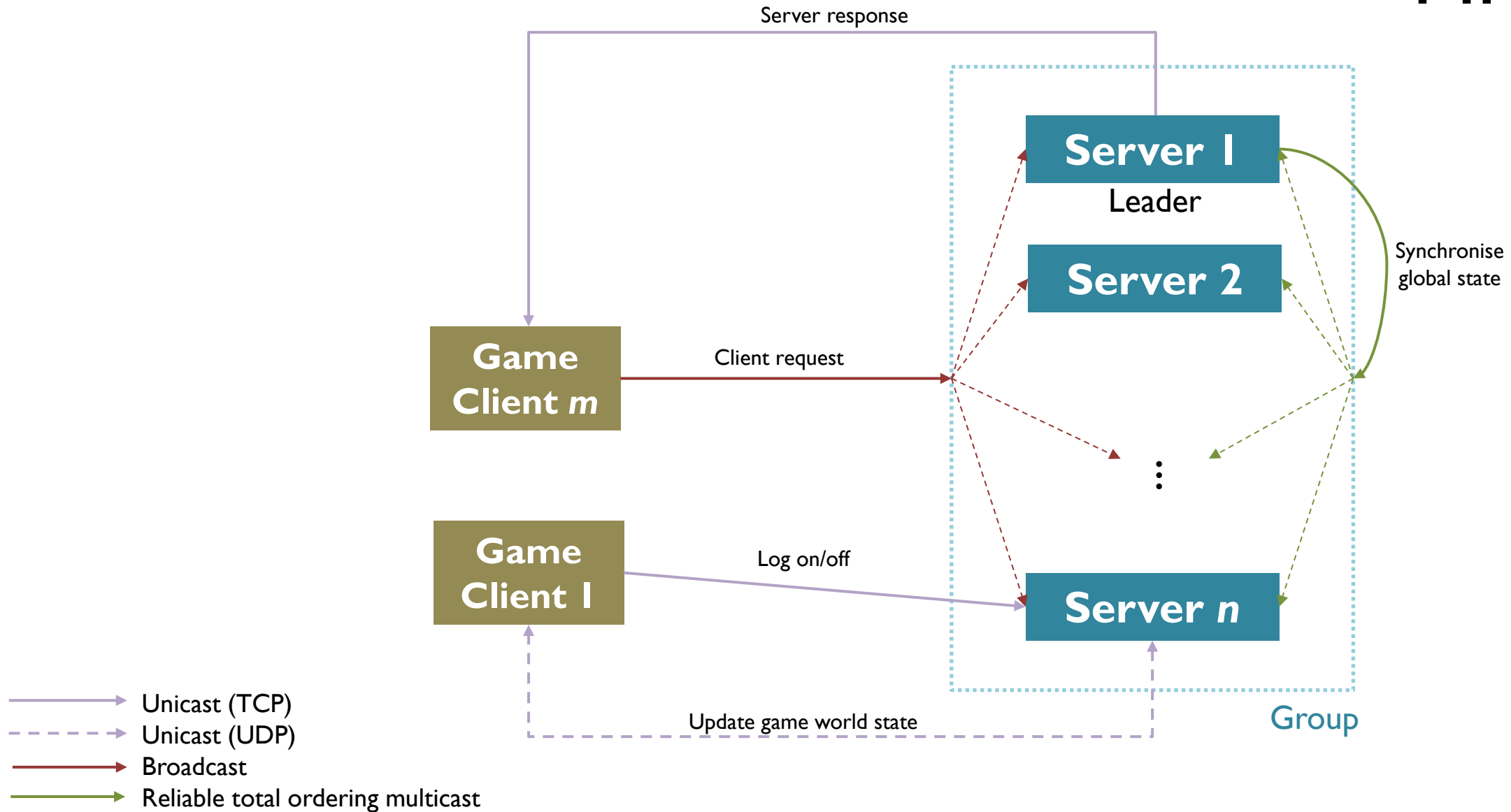
Architectures

- One server, one client
 - Toy model
 - Useful for explanations
 - Useless for real applications
- One server, multiple clients
 - Simplistic model
 - Useful for limited types of applications
- Multiple (and multiple types of) servers, multiple clients
 - Realistic model
 - Typical for distributed applications

Chat application



MMOG

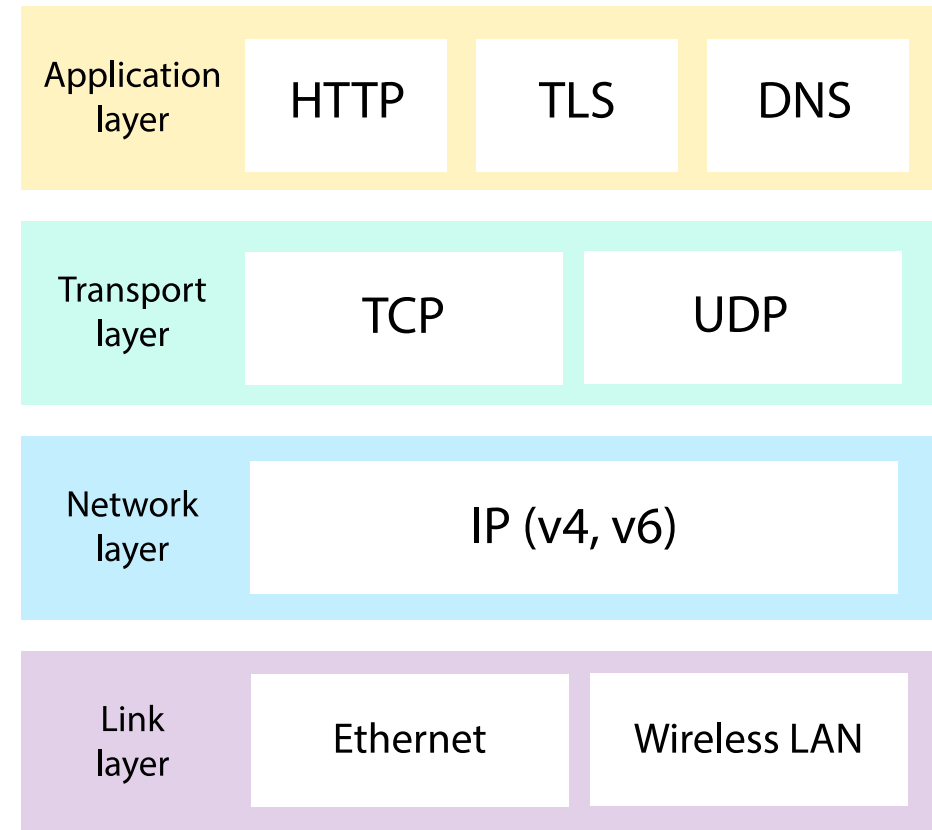


How to establish a connection between a client and a server?

SOCKETS AND PROTOCOLS

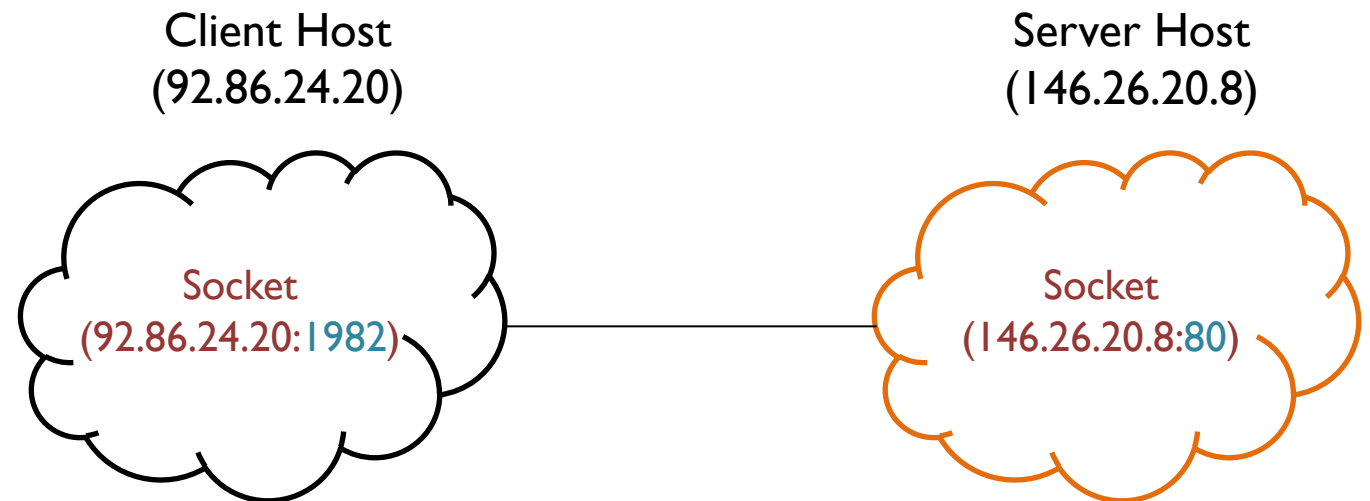
Internet Protocol Suite

- Host (network layer)
 - IP address of the machine
- Port (transport layer)
 - Port number where the application runs



Sockets

- Create and manage connections
- *Send* and *receive* data through connections
- OS-level data structure
 - Remote IP address
 - Remote port number
 - Local IP address
 - Local port number
 - Message queue



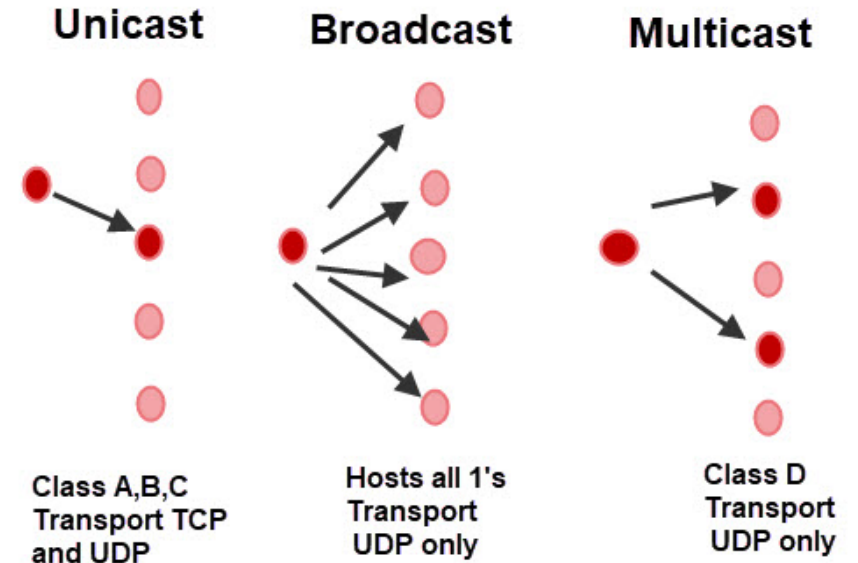
Communication protocols (summary)

- **Transmission Control Protocol (TCP)**

- Requires establishing a connection
- Transmits byte stream
- Keeps track of lost data and resend lost data (reliability at the transport layer)
- Uses sequence numbers to order arriving data (ordering at the transport layer)
- Relative slow
- Only unicast

- **User Datagram Protocol (UDP)**

- Connectionless
- Transmits datagrams
- Does not care about reliability
- Does not care about arrival order of datagrams
- Relatively fast
- Unicast and (IP) multicast



Sockets in Python

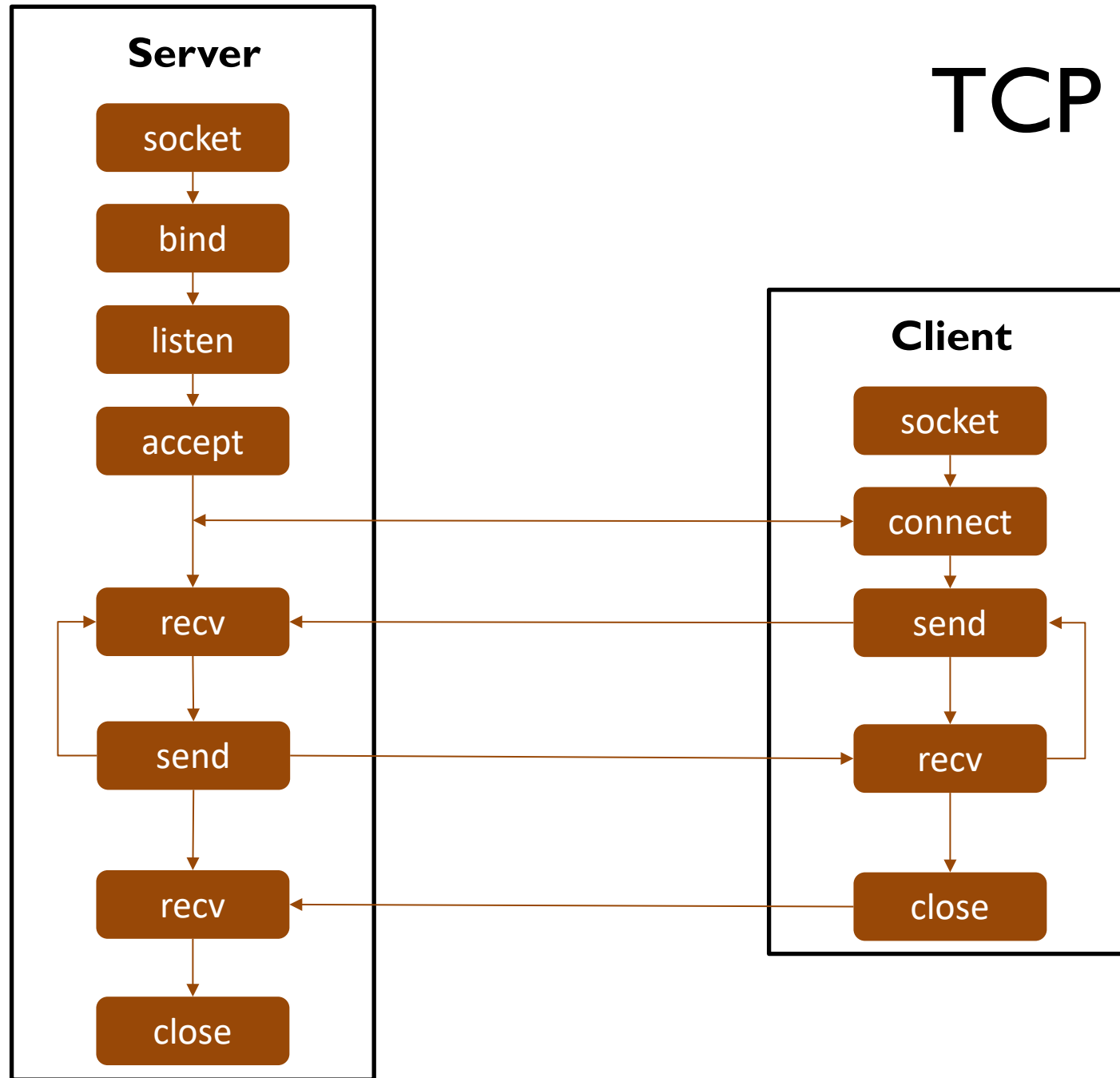
- **socket module**
- **TCP and UDP sockets**
- **Basic socket API functions**
 - `socket(family, type)`
 - `bind(address)`
 - `listen()`
 - `accept()`
 - `connect(address)`
 - `send(bytes)`
 - `recv(bufsize)`
 - `close()`

- **Family**
 - `AF_INET`
 - `AF_INET6`
 - `AF_UNIX`
 - ...
- **Type**
 - `SOCK_STREAM` for TCP
 - `SOCK_DGRAM` for UDP

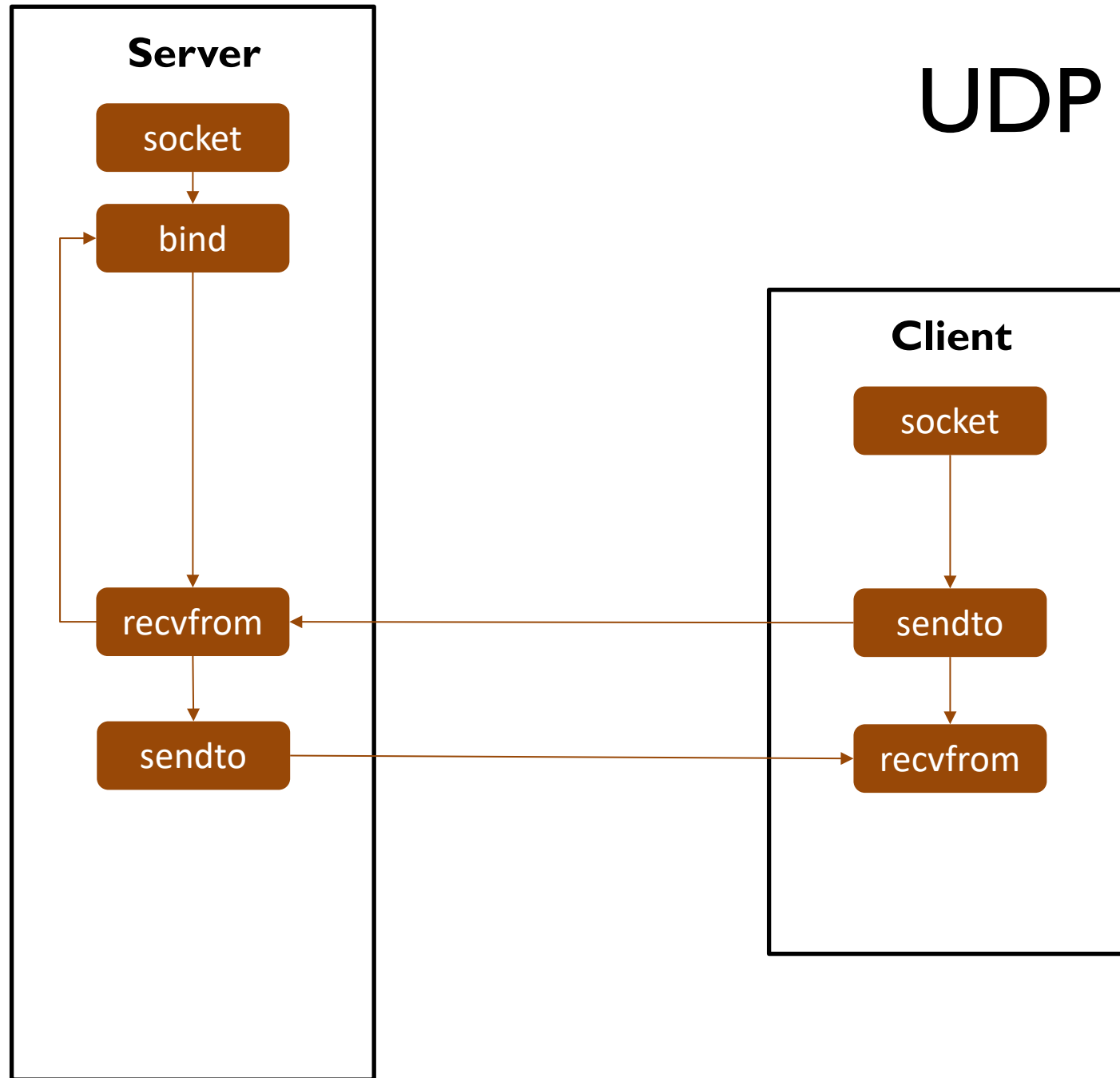
- **Send**
 - `send`
 - `sendall`
 - `sendto`

- **Receive**
 - `recv`
 - `recvfrom`

TCP data flow



UDP data flow



With sockets

ONE SERVER, ONE CLIENT

Server in Python

```
import socket

# Create a UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Server application IP address and port
server_address = '127.0.0.1'
server_port = 10001

# Buffer size
buffer_size = 1024

message = 'Hi client! Nice to connect with you!'

# Bind socket to port
server_socket.bind((server_address, server_port))
print('Server up and running at {}:{}'.format(server_address, server_port))

while True:
    print('\nWaiting to receive message...\n')

    data, address = server_socket.recvfrom(buffer_size)
    print('Received message from client: ', address)
    print('Message: ', data.decode())

    if data:
        server_socket.sendto(str.encode(message), address)
        print('Replied to client: ', message)
```

Client in Python

```
import socket

# Create a UDP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Bind the socket to the port
server_address = '127.0.0.1'
server_port = 10001

# Buffer size
buffer_size = 4096

message = 'Hi server!'

# Send data
client_socket.sendto(message.encode(), (server_address, server_port))
print('Sent to server: ', message)

# Receive response
print('Waiting for response...')
data, server = client_socket.recvfrom(buffer_size)
print('Received message from server: ', data.decode())
```

```
# For TCP sockets
try:
    # Send data
    ...

    # Receive response
    ...
finally:
    client_socket.close()
    print('Socket closed')
```

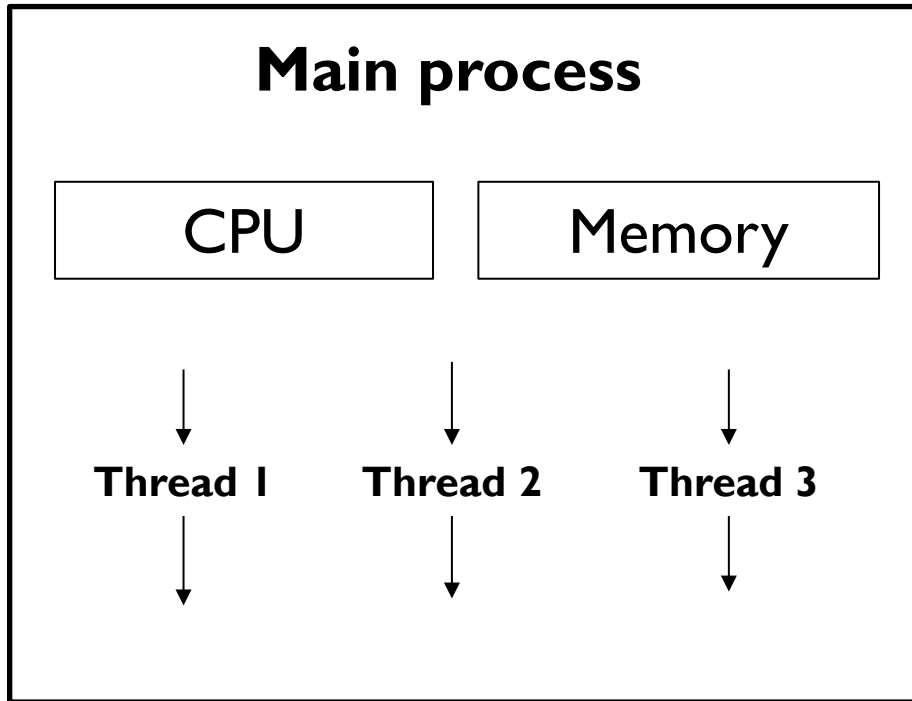
MULTIPROCESSING

Concurrency

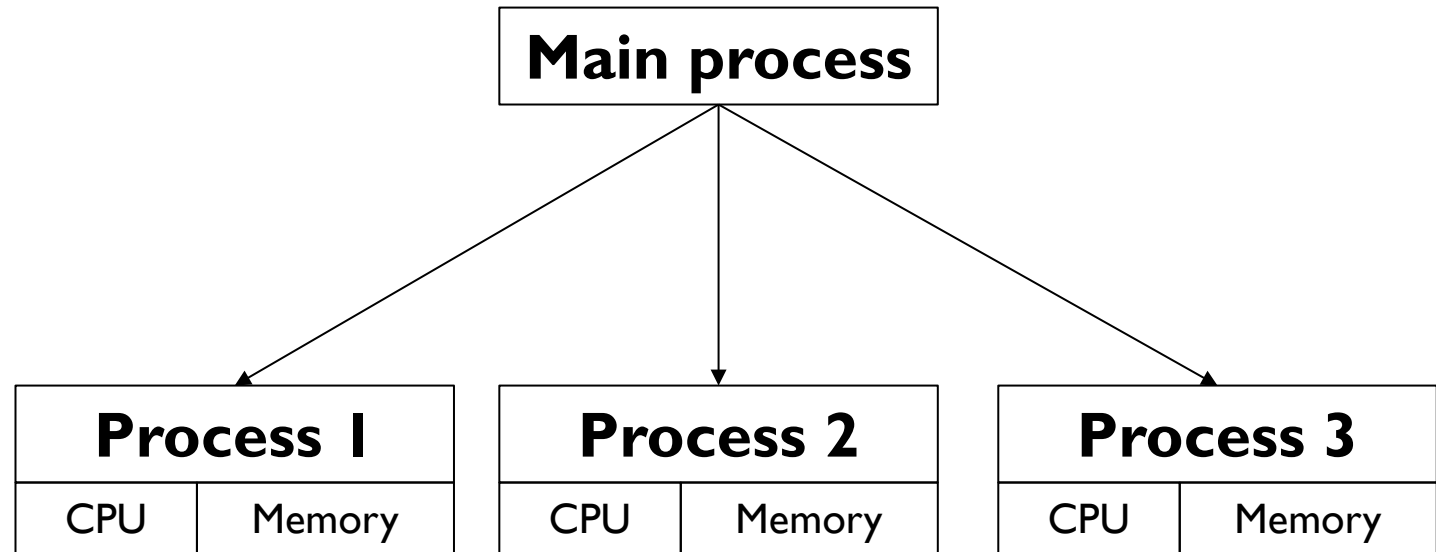
- **Program**
 - Source code for process(es)
- **Process**
 - Unit of program execution as seen by an OS
- **Thread**
 - Sequential flow of control within a process
 - Process can contain one or more threads
- **Multiprocessing**
 - Concurrent execution of several programs on one machine
- **Multithreading**
 - Execution of a program with multiple threads

Concurrency

Multithreading



Multiprocessing



Concurrency in Python

- Threads (`threading`)
- Processes (`multiprocessing`)
- Asynchronous I/O (`asyncio`)

Model	Execution unit	Concurrency	Parallelism	Memory space	Scales across CPU cores	Overhead
<code>threading</code>	Threads (within one process)	Yes (multiple threads share one process)	No (limited by GIL)	Shared between threads	No	Low
<code>multiprocessing</code>	Processes (separate OS processes)	Yes (multiple processes execute independently)	Yes (each process runs on its own CPU core)	Isolated per process	Yes	High
<code>asyncio</code>	Coroutines (within one event loop)	Yes (cooperative multitasking via coroutines)	No (runs on a single thread)	Shared (single-threaded)	No	Very low

Multiprocessing in Python

- Local and “remote” concurrency [1]
 - Local: run multiple processes on the same machine, each in its own memory space
 - “Remote”: run multiple processes in different memory spaces, which may be on the same machine or across machines if configured
- Spawning processes
 - `Process` class, `Pool` for managing a group of processes
- Communication between processes
 - Queues, Pipes, Managers, Shared Memory
- Synchronisation between processes
 - Standard synchronisation primitives (e.g., locks)

Process class

- `Process(target, name, args)`
 - `run()`
 - `start()`
 - `join()`

Processes in Python

```
from multiprocessing import Process
import os

def salute(course):
    print('Hello', course)
    print('Parent process id:', os.getppid())
    print('Process id:', os.getpid())

if __name__ == '__main__':
    p = Process(target=salute, args=('DS',))
    p.start()
    p.join()
```

With sockets

**ONE SERVER, MULTIPLE CLIENTS AS
PROCESSES ON A SINGLE MACHINE**

Example

Develop a client-server application that supports concurrency via multiprocessing. Three client processes send a “Hello” message to the server, including their process ID. The server receives such a message and spawns a process that sends back a “Hello” message with its own process ID.

Multiple client processes

```
import socket
import multiprocessing
import os

def send_message(s_address, s_port):
    # Create a UDP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Message sent to server
    message = 'Hi from ' + str(os.getpid())

    # Send data
    client_socket.sendto(str.encode(message), (s_address, s_port))
    print('Sent to server: ', message)

    # Receive response from server
    print('Waiting for response...')
    data, server = client_socket.recvfrom(1024)
    print('Received message: ', data.decode())

if __name__ == '__main__':
    # Server application IP address and port
    server_address = '127.0.0.1'
    server_port = 10001

    for i in range(3):
        # Spawn three client processes
        p = multiprocessing.Process(target=send_message, args=(server_address, server_port))
        p.start()
        p.join
```

Multiple client processes

```
import socket
import multiprocessing
import os

def send_message(s_address, s_port):
    # Create a UDP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Message sent to server
    message = 'Hi from ' + str(os.getpid())

    # Send data
    client_socket.sendto(str.encode(message), (s_address, s_port))
    print('Sent to server: ', message)

    # Receive response from server
    print('Waiting for response...')
    data, server = client_socket.recvfrom(1024)
    print('Received message: ', data.decode())

if __name__ == '__main__':
    # Server application IP address and port
    server_address = '127.0.0.1'
    server_port = 10001

    # Spawn three client processes
    prcs = [multiprocessing.Process(target=send_message, args=(server_address, server_port))
            for _ in range(3)]
    for p in prcs: p.start()
    for p in prcs: p.join()
```

Run concurrently

**MULTIPROCESS SERVERS, MULTIPLE
CLIENTS**


```
import multiprocessing
import socket
import os
```

```
class Server(multiprocessing.Process):
    def __init__(self, server_socket, received_data, client_address):
        super(Server, self).__init__()
        self.server_socket = server_socket
        self.received_data = received_data
        self.client_address = client_address

    # Override run method (check Python's multiprocessing documentation)
    def run(self):
        # Message to be sent to client
        message = 'Hi ' + self.client_address[0] + ':' + str(self.client_address[1]) + '. This is server with process ID' + str(os.getpid())
        # Send message to client
        self.server_socket.sendto(str.encode(message), self.client_address)
        print('Sent to client: ', message)

if __name__ == "__main__":
    # Create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Server application IP address and port
    server_address = '127.0.0.1'
    server_port = 10002

    # Buffer size
    buffer_size = 1024

    # Bind socket to address and port
    server_socket.bind((server_address, server_port))
    print('Server up and running at {}:{}'.format(server_address, server_port))

    while True:
        # Receive message from client
        data, address = server_socket.recvfrom(buffer_size)
        print('Received message \'{}\' at {}'.format(data.decode(), address[0], address[1]))
        # Create a server process
        p = Server(server_socket, data, address)
        p.start()
        p.join()
```

Multiprocessing server

```
import multiprocessing
import socket
import os
```

```
class Server(multiprocessing.Process):
    def __init__(self, server_socket, received_data, client_address):
        super(Server, self).__init__()
        self.server_socket = server_socket
        self.received_data = received_data
        self.client_address = client_address

    # Override run method (check Python's multiprocessing documentation)
    def run(self):
        # Message to be sent to client
        message = 'Hi ' + self.client_address[0] + '!' + str(self.client_address[1]) + '. This is server with process ID' + str(os.getpid())
        # Send message to client
        self.server_socket.sendto(str.encode(message), self.client_address)
        print('Sent to client: ', message)

if __name__ == "__main__":
    # Create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Server application IP address and port
    server_address = '127.0.0.1'
    server_port = 10002

    # Buffer size
    buffer_size = 1024

    # Bind socket to address and port
    server_socket.bind((server_address, server_port))
    print('Server up and running at {}:{}'.format(server_address, server_port))

    while True:
        # Receive message from client
        data, address = server_socket.recvfrom(buffer_size)
        print('Received message \'{}\'' at {}:{}'.format(data.decode(), address[0], address[1]))
        # Create a server process
        p = Server(server_socket, data, address)
        p.start()
        p.join()
```

Multiprocessing server

Three improvements:

- Let each worker process create its own UDP socket to send a reply
- Handle multiple requests concurrently (no join())
- Mark worker processes daemon so they exit with the parent (p.daemon = True)

Exercise

Develop a client-server application that supports asynchronous concurrency. The server should be able to receive requests from multiple clients at the same time. A client sends a number and receives back the square of that number.

Considerations

Architecture & Communication

- What architecture model will your system be based on?
 - Client-server? Peer-to-peer?
 - Do **not** restrict the number of components in your model – design the system so it can scale to an arbitrary number of clients/servers/peers
- What types of communication do you envision between system components and/or groups of components?
 - TCP unicast? UDP unicast? IP multicast? Reliable ordered multicast?
 - Sockets are needed for *all* of them

Considerations

Process/Thread Design

- Separate responsibilities
 - Use dedicated processes/threads for specific roles (e.g., request handling, discovery, heartbeat)
 - Keep each process/thread simple and focused
- Supervision and lifecycle
 - Let a main (supervisor) process spawn and monitor worker processes/threads
 - Avoid calling `join()` inside request loops; use it at program shutdown

Considerations

Sockets & Hostnames

- Using sockets
 - Avoid sharing a bound socket between processes; each worker should create its own if needed
 - UDP sockets can be reused or closed after use
 - TCP sockets **must** be closed after the application completes the relevant work
 - Add timeouts and handle exceptions
- Using hostnames
 - *If you use a hostname in the host portion of IPv4/v6 socket address, the program may show a non-deterministic behaviour [...]. ([Source](#))*
 - Use IP addresses instead (e.g., 127.0.0.1 for localhost)

Considerations

Testing & Debugging

- Test your architecture
 - Use several terminal instances (run one or more instances of the server/client program concurrently)
 - Optionally use virtual machines to run client/server programs
 - Use a second network to run server/client programs
- If something does not go as expected
 - Monitor network and/or socket states (e.g., use `netstat`, `lsof`, `Wireshark`)
 - If processes or threads do not behave as expected, add logging of process IDs or thread names
 - Read tutorials; read the documentation of the socket module/library
 - Check your network settings (e.g., disable/adjust firewall, router, etc.)
 - Debug your programs (e.g., test progressively - start with one client and one server, then scale up)